

CU-AIM Server User Manual 2026

A practical CU-AIM server guide for our lab users

Link for related codes: [github_placeholder](#)

Last Updated: 11/09/2026

Created by Peeradon Sarnkaew

Contact Email: peeradonsarnkaew46@gmail.com

Contents

1	Prepare First-Time Access	1
1.1	Connect to the Chula VPN	1
1.2	Create an SSH Key Pair	1
1.3	Request Account Authorization	4
1.4	Optional: Transfer Files with Cyberduck	5
2	Connect to CU-AIM	6
2.1	Connect from a Terminal	6
2.2	Install a Compatible VS Code Version	6
2.3	Configure Remote-SSH in VS Code	7
3	Plan Project Storage and Resources	8
3.1	Use the Assigned Project Directory	8
3.2	Check Account and Scheduling Limits	8
3.3	Inspect Partitions and Compute Nodes	10
4	Run and Monitor Slurm Jobs	11
4.1	Understand a Slurm Job	11
4.2	Choose a Submission Method	11
4.3	Cancel a Job	13
4.4	Prepare the Software Environment	13
4.5	Monitor Jobs and the Queue	15
5	Coordinate Multiple Slurm Workloads	18
5.1	Distinguish Jobs, Tasks, and Job Steps	18
5.2	Choose a Multi-Program Structure	18
5.3	Run a Numbered Series of Programs	20
6	Use Singularity Containers	24
6.1	Obtain a Container Image	24
6.2	Create a Python Environment in the Container	25
6.3	Run a Container in a Batch Job	26
6.4	Run a Container Interactively	27

1 Prepare First-Time Access

This manual introduces the routine workflow for using the CU-AIM computing cluster. A cluster is a shared group of computers managed by a scheduler, which assigns resources to submitted work. The manual is intended for students who are familiar with basic files and Python but are new to remote servers and Slurm.

Complete this chapter once on each computer that you will use to access CU-AIM. The normal order is to connect to the university VPN, create an SSH key pair, submit the public key for authorization, and then test the connection.

1.1 Connect to the Chula VPN

1. Download and install Cisco AnyConnect by following the Chulalongkorn University IT instructions: <https://www.it.chula.ac.th/en/service/en-vpn/>.
2. Open Cisco AnyConnect and enter the VPN server given in the current university instructions. Sign in with your Chula username and password. **You have to connect to this VPN every time you want to use the CUAIM server!**

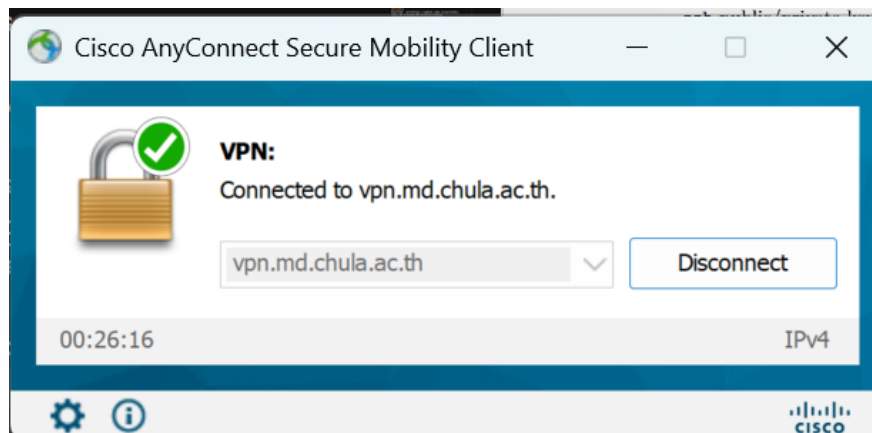


Figure 1.1: Cisco AnyConnect VPN connection settings.

1.2 Create an SSH Key Pair

CU-AIM uses SSH public-key authentication. An SSH key pair contains a public key that you may share and a private key that must remain on your computer. Never send your private key to another

person or store it in a shared project directory.

Choose either the PuTTY method or the terminal method below. You do not need to use both.

1.2.1 Generate a Key with PuTTY

The walkthrough video at <https://youtu.be/atx3VhDU0r8> demonstrates this process. The essential steps are summarized here.

1. Download and install PuTTY from <https://www.putty.org/>.
2. Open PuTTYgen, select the settings shown in Figure 1.2, and click **Generate**.

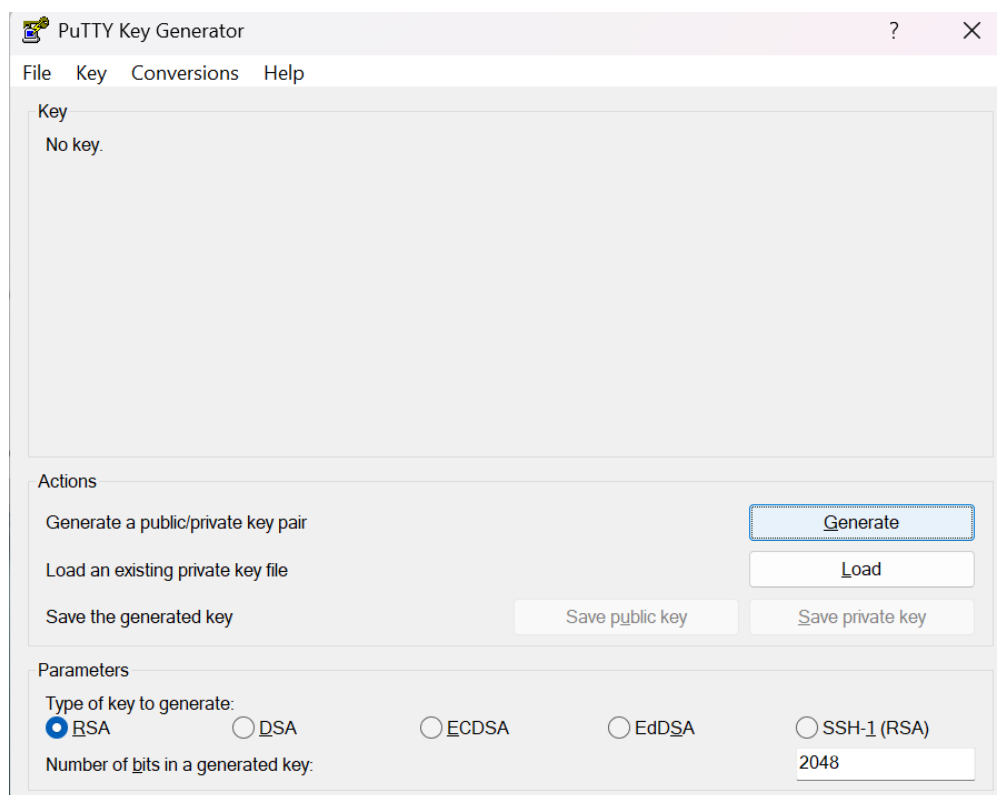


Figure 1.2: PuTTYgen key-generation settings.

3. Move the pointer over the blank area until PuTTYgen finishes creating the key.
4. Save the public and private keys in a secure location on your computer. Protect the private key with a passphrase.
5. Open PuTTY and select the private key file as shown in Figure 1.5.

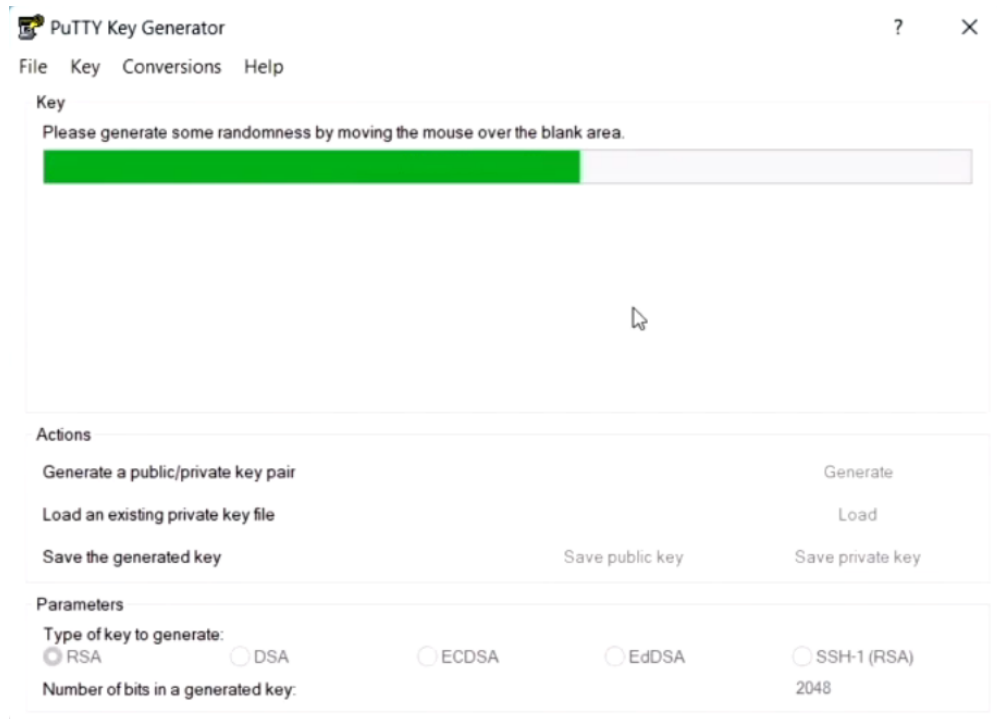


Figure 1.3: PuTTYgen generating an SSH key.

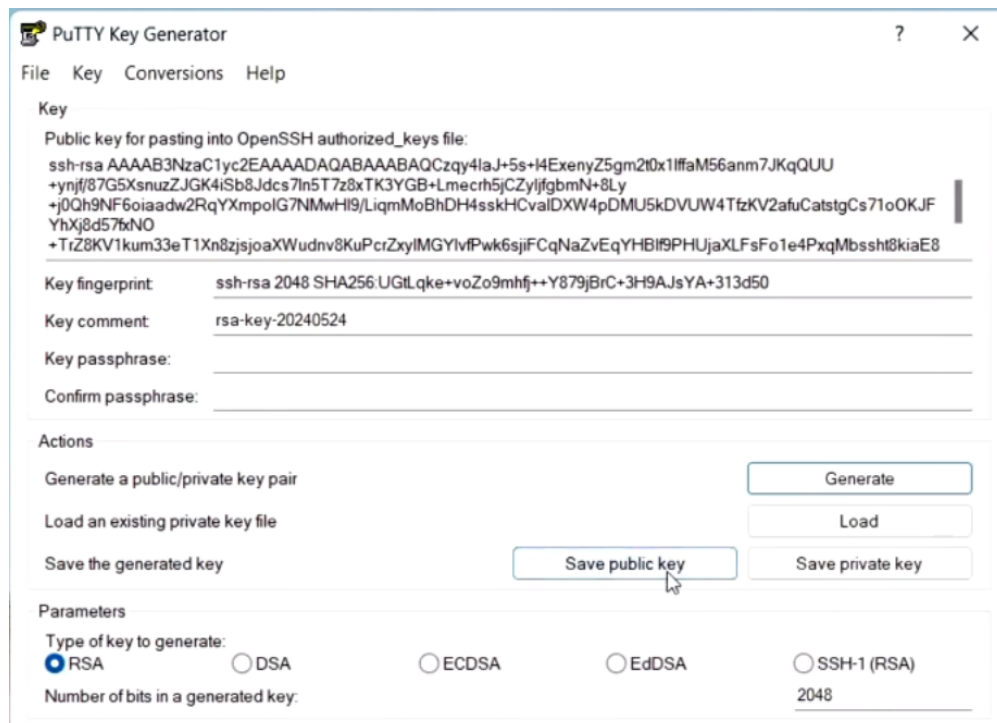


Figure 1.4: Saving the generated public and private keys.

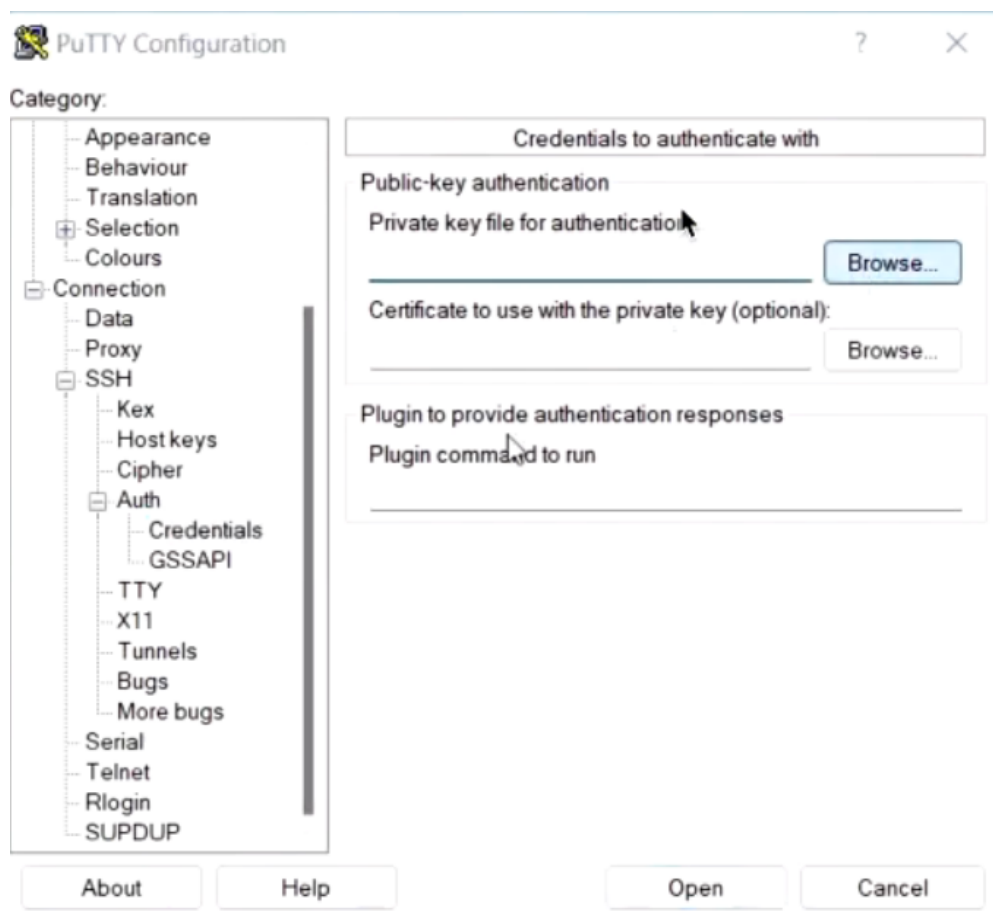


Figure 1.5: Selecting a private key in PuTTY.

1.2.2 Generate a Key in a Terminal

Open a terminal and run the following command:

```
ssh-keygen -t rsa -b 4096 -f ~/.ssh/id_rsa_cuaim -C "abc@example.com"
```

This command creates a 4096-bit RSA key pair. The private key is stored at the specified path, and the public key is stored at the same path with the suffix `.pub`. The email address is only a label for identifying the key. When prompted, enter a passphrase to protect the private key.

1.3 Request Account Authorization

Send only the public key file, normally the file ending in `.pub`, to your professor or designated cluster contact. Wait for confirmation that the key has been added to your CU-AIM account before connecting.

1.4 Optional: Transfer Files with Cyberduck

Cyberduck provides a graphical interface for transferring files on Windows and macOS. It is optional because files can also be managed through a terminal or VS Code. Install it from <https://cyberduck.io/download/>; the walkthrough video at <https://youtu.be/atx3VhDU0r8> shows the CU-AIM configuration.

Open Cyberduck, click the “+” button, enter the assigned server details, and select your SSH private key as shown in Figure 1.6.

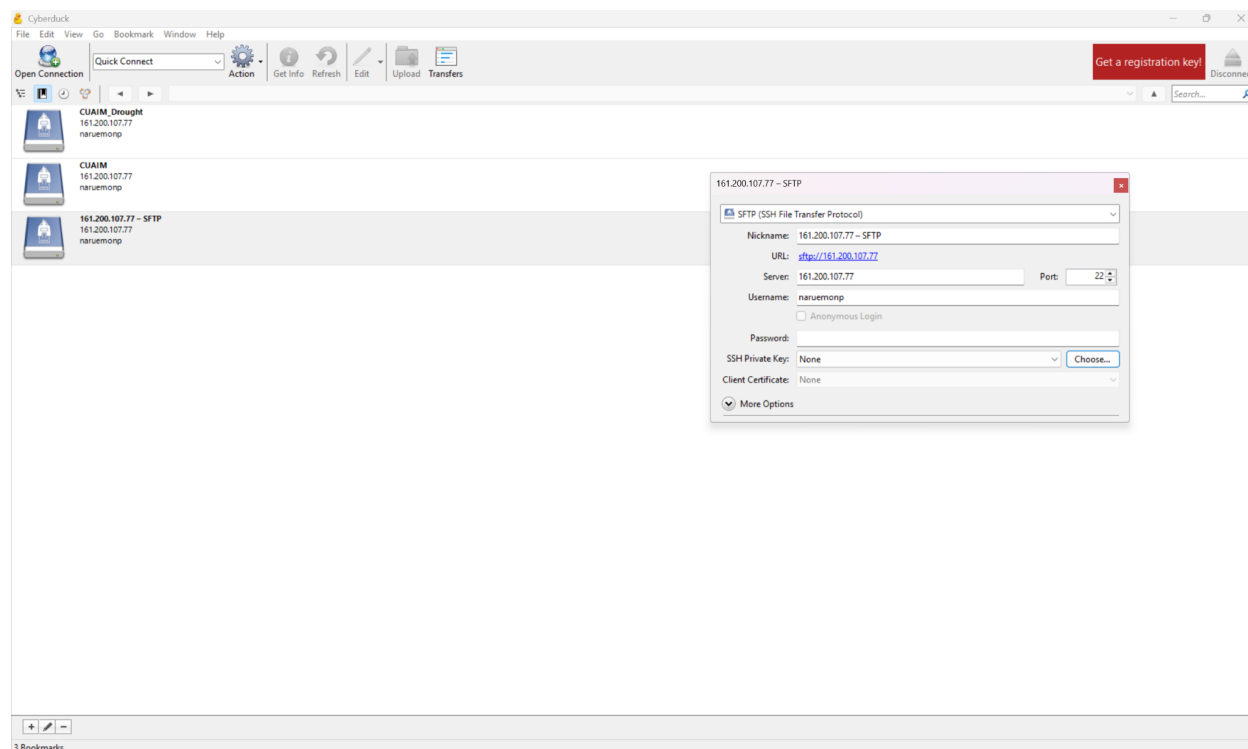


Figure 1.6: Cyberduck connection settings.

After connecting, open your assigned project directory. **Do not assume that `/data/home` is suitable for project data** because it may have a small storage quota. Confirm the correct path with your supervisor. Cyberduck is convenient for occasional drag-and-drop transfers; a terminal or VS Code is generally more efficient for repeated or large transfers.

2 Connect to CU-AIM

Each work session begins by connecting to the Chula VPN as described in Section 1.1. Keep the SSH private key on your local computer and confirm that your public key has been authorized.

2.1 Connect from a Terminal

Open a local terminal and use SSH to connect:

```
ssh naruemonp@161.200.107.77 -i </path/to/your/private_key>
```

A successful connection opens a shell on a CU-AIM login node. This entry computer is intended for file management, software setup, and Slurm submission commands. Submit computational work to Slurm instead of running it directly on the login node. Enter `exit` or `Ctrl + D` when you are ready to close the session. If the connection fails, first confirm the VPN connection, server address, username, key path, and account authorization.

2.2 Install a Compatible VS Code Version

VS Code with the Remote-SSH extension provides a graphical editor and terminal for remote work. However, the current version of VS Code cannot be used with the CUAIM server because of incompatibilities with the system libraries on CU-AIM.

1. [Download VS Code 1.85.2](#)

Fortunately, you do not need to uninstall your current version. Instead, download the compatible version above, extract the archive, and run `Code.exe` from the extracted directory whenever you need to access the CUAIM server. This release is older than the current version, so some recent extensions may not support it. You should also disable automatic updates for this installation and keep a separate, up-to-date installation for other work.

2. Then, install the **Remote-SSH** extension in this VS Code installation. If it does not load, install an extension version that supports the tested VS Code release.

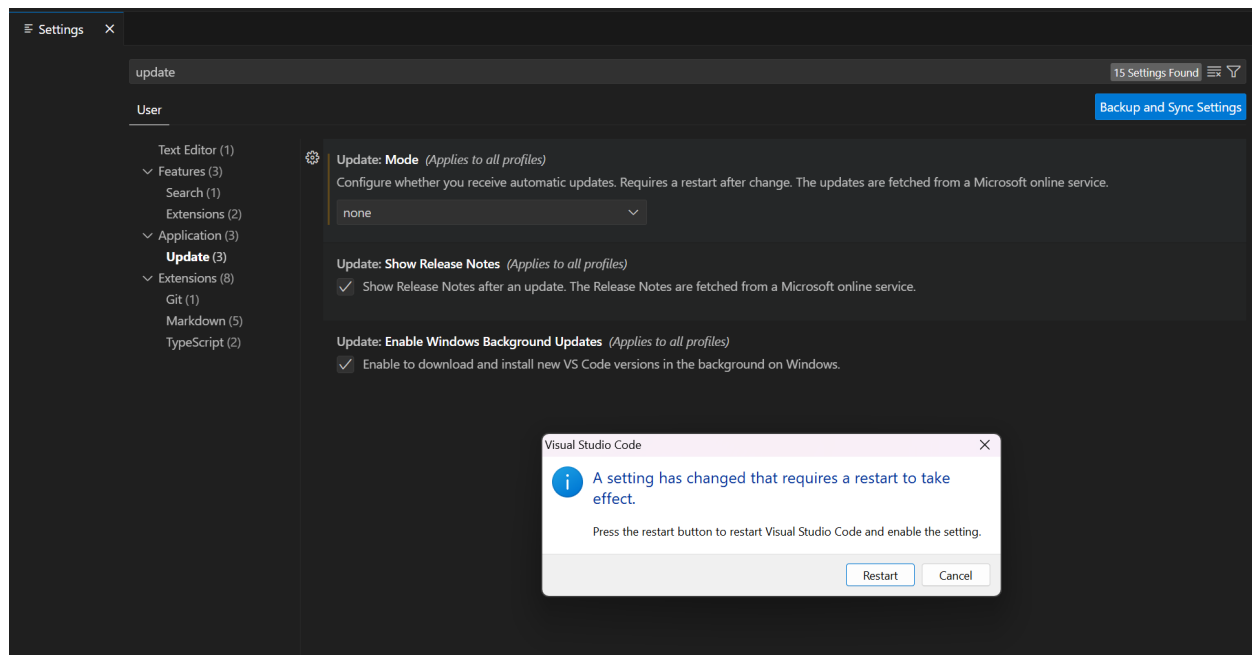


Figure 2.1: Disabling automatic updates in the tested VS Code installation.

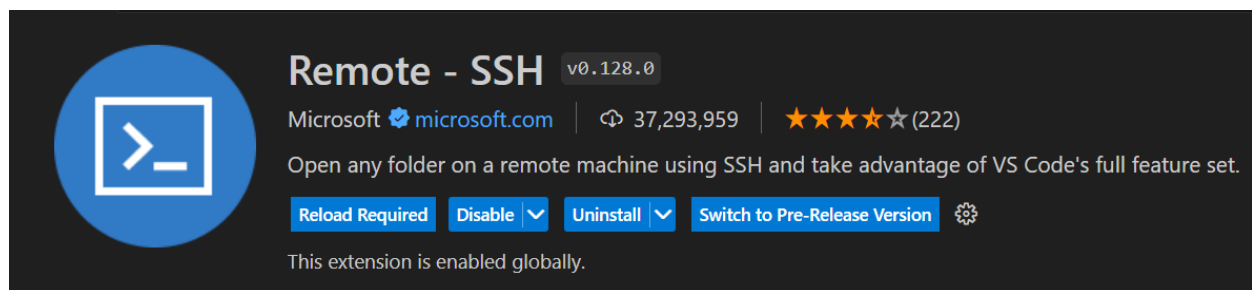


Figure 2.2: The Remote-SSH extension in VS Code.

2.3 Configure Remote-SSH in VS Code

1. Open VS Code, press **Ctrl+Shift+P**, and select **Remote-SSH: Open SSH Configuration File**. Choose the user configuration file, normally `%USERPROFILE%/.ssh/config` on Windows.
2. Add a host entry and replace each placeholder with your assigned value:

```
Host cuaim
  HostName 161.200.107.77
  User naruemomp
  IdentityFile /path/to/your/private_key
```

To connect, press **Ctrl+Shift+P**, select **Remote-SSH: Connect to Host**, choose `cuaim`, and select **Linux** if prompted. A new VS Code window will open for the remote session. The next chapter explains how to open the correct project directory.

3 Plan Project Storage and Resources

Before running a program, open the correct project directory and identify the resources that your account is permitted to request. This prevents common problems such as exceeding a storage quota or submitting a job with an unavailable QoS.

3.1 Use the Assigned Project Directory

After connecting, your session may begin in `/data/home/`. This home directory is suitable for small personal configuration files but may have limited storage. In VS Code, select **File > Open Folder** and enter your assigned project path, for example:

```
/data/project/your_project
```

Create a personal project folder within the assigned directory. If your research group has not specified a convention, use a descriptive name such as:

```
{firstname}_{first character of the surname}_{project_name}
```

Keep source code, data, environments, model outputs, and logs within the approved project storage. Do not place large project files in `/data/home/`.

3.2 Check Account and Scheduling Limits

CU-AIM uses Slurm to assign computing resources. A *Quality of Service* (QoS) is a named policy that can limit requested CPUs, GPUs, memory, and running time. A partition is a group of compute nodes used for a particular class of work. Your account must be allowed to use both the requested partition and QoS.

3.2.1 Check the Available QoS Options

Run the CU-AIM helper command in a remote terminal:

```
check_qos
```

The output shows the QoS options and limits currently available to your account. Use this output as the authoritative source when it differs from this manual.

```
(code-server-env) [naruemonp@cuaim-frontend1 naruemon]$ check_qos
sacctmgr show qos format=name%15,MaxTRESPU%90,MaxWall
```

Name	MaxTRESPU	MaxWall
normal		
chics		
ekapol		
voraphoj		
pornchai		
thanac		
cuaim		
gpu40g	cpu=16,gres/gpu:1g.5gb=0,gres/gpu:2g.10gb=0,gres/gpu:3g.20gb=0,gres/gpu:7g.40gb=1,mem=64G	8-00:00:00
gpu20g	cpu=8,gres/gpu:1g.5gb=0,gres/gpu:2g.10gb=0,gres/gpu:3g.20gb=1,gres/gpu:7g.40gb=0,mem=32G	8-00:00:00
gpu10g	cpu=4,gres/gpu:1g.5gb=0,gres/gpu:2g.10gb=1,gres/gpu:3g.20gb=0,gres/gpu:7g.40gb=0,mem=16G	4-00:00:00
gpu5g	cpu=2,gres/gpu:1g.5gb=1,gres/gpu:2g.10gb=0,gres/gpu:3g.20gb=0,gres/gpu:7g.40gb=0,mem=8G	4-00:00:00
cpu32	cpu=32,mem=8G	4-00:00:00
cpu16	cpu=16,mem=8G	4-00:00:00
cpu8	cpu=8,mem=8G	4-00:00:00
cpu4	cpu=4,mem=8G	4-00:00:00
cpu24h	cpu=24,mem=1T	4-00:00:00
ngs16	cpu=16,mem=96G	8-00:00:00
ngs8	cpu=8,mem=48G	8-00:00:00
studentgpu	cpu=2,gres/gpu:1g.5gb=1,gres/gpu:2g.10gb=0,gres/gpu:3g.20gb=0,gres/gpu:7g.40gb=0,mem=8G	2-00:00:00
studentcpu	cpu=4,mem=16G	2-00:00:00
interactive-cpu	cpu=2,mem=8G	02:00:00
interactive-gpu	cpu=4,gres/gpu:1g.5gb=1,gres/gpu:2g.10gb=0,gres/gpu:3g.20gb=0,gres/gpu:7g.40gb=0,mem=16G	02:00:00
alphafold-v1	cpu=16,gres/gpu:1g.5gb=0,gres/gpu:2g.10gb=0,gres/gpu:3g.20gb=4,gres/gpu:7g.40gb=0,mem=32G	8-00:00:00
gpu10gh	cpu=16,gres/gpu:1g.5gb=0,gres/gpu:2g.10gb=1,gres/gpu:3g.20gb=0,gres/gpu:7g.40gb=0,mem=64G	4-00:00:00
gpu20gh	cpu=16,gres/gpu:1g.5gb=0,gres/gpu:2g.10gb=0,gres/gpu:3g.20gb=1,gres/gpu:7g.40gb=0,mem=64G	8-00:00:00
cpu40h	cpu=40,mem=1T	4-00:00:00
gpu40g64	cpu=64,gres/gpu:1g.5gb=0,gres/gpu:2g.10gb=0,gres/gpu:3g.20gb=0,gres/gpu:7g.40gb=1,mem=128G	8-00:00:00
gpu40g24m	cpu=24,gres/gpu:1g.5gb=0,gres/gpu:2g.10gb=0,gres/gpu:3g.20gb=0,gres/gpu:7g.40gb=2,mem=128G	

Figure 3.1: Example QoS information displayed by CU-AIM.

3.2.2 Check the Account Association

Access may also depend on your research group or Slurm account. Display the group information assigned to your user:

```
check_group
```

Table 3.1 records the QoS values available when this manual was prepared. These cluster settings may change, so verify them before every resource-intensive submission.

Table 3.1: Documented CU-AIM QoS limits

QoS	CPUs	GPU resource	Memory	Maximum duration
cpu40h	40 CPUs	CPU only	1 TB	4 days
interactive-gpu	4 CPUs	1 × 1g.5gb	16 GB	2 hours
gpu10gh	16 CPUs	1 × 2g.10gb	64 GB	4 days
gpu20gh	16 CPUs	1 × 3g.20gb	64 GB	8 days
gpu40g	16 CPUs	1 × 7g.40gb	64 GB	8 days
gpu40g24m	24 CPUs	2 × 7g.40gb	128 GB	8 days

The following standard Slurm accounting command provides another view of the account and QoS associations for a user:

```
sacctmgr show assoc user=naruemonp format=account,user%15,qos%50
```

3.3 Inspect Partitions and Compute Nodes

The scheduler runs jobs on compute nodes within a partition. Use `sinfo` to inspect node states and advertised resources:

```
sinfo -o "%.15R %.8D %.10m %.5c %7z %60G %N"
```

```
(base) [naruemonp@cuaaim-frontend1 peeradon_drought]$ sinfo -N -e -o "%.15R %.8D %.10m %.5c %7z %60G %N"
PARTITION    NODELIST      STATE    CPUS    CPUS(A/I/O/T)  MEMORY    FREE_MEM    GRES
interactive-gpu  cuaaim-dgx-01  mixed    128     76/52/0/128    860000    830560      gpu:7g.40gb:5,gpu:3g.20gb:6
gpu            cuaaim-dgx-01  mixed    128     76/52/0/128    860000    830560      gpu:7g.40gb:5,gpu:3g.20gb:6
interactive-cpu  cuaaim-fat-01  mixed    72      27/45/0/72     690000    535894      (null)
cpu*           cuaaim-fat-01  mixed    72      27/45/0/72     690000    535894      (null)
cpu*           cuaaim-fat-02  idle     224     0/224/0/224    3030000   2937921     (null)
interactive-cpu  cuaaim-fat-02  idle     224     0/224/0/224    3030000   2937921     (null)
```

Figure 3.2: Example Slurm node and partition information.

The output summarizes partitions, node states, memory, CPUs, and configured resources. To inspect one node in more detail, use:

```
scontrol show node <node-name>
```

```
(base) [naruemonp@cuaaim-frontend1 peeradon_drought]$ scontrol show node cuaaim-dgx-01
NodeName=cuaaim-dgx-01 Arch=x86_64 CoresPerSocket=64
CPUAlloc=44 CPUTot=128 CPULoad=9.98
AvailableFeatures=(null)
ActiveFeatures=(null)
Gres=gpu:7g.40gb:5,gpu:3g.20gb:6
NodeAddr=cuaaim-dgx-01 NodeHostName=cuaaim-dgx-01 Version=21.08.5
OS=Linux 5.4.0-159-generic #176-Ubuntu SMP Mon Aug 14 12:04:20 UTC 2023
RealMemory=860000 AllocMem=139264 FreeMem=830732 Sockets=2 Boards=1
State=MIXED ThreadsPerCore=1 TmpDisk=0 Weight=1 Owner=N/A MCS_label=N/A
Partitions=gpu,interactive-gpu
BootTime=2026-08-03T09:11:12 SlurmdStartTime=2026-09-07T09:59:54
LastBusyTime=2026-09-09T10:54:44
CfgTRES=cpu=128,mem=860000M,billing=128,gres/gpu:3g.20gb=6,gres/gpu:7g.40gb=5
AllocTRES=cpu=44,mem=136G,gres/gpu:7g.40gb=3
CapWatts=n/a
CurrentWatts=0 AveWatts=0
ExtSensorsJoules=n/s ExtSensorsWatts=0 ExtSensorsTemp=n/s
```

Figure 3.3: Detailed information for a Slurm node.

Do not request a particular node unless the CU-AIM documentation or a cluster administrator instructs you to do so. A narrow node request can keep an otherwise valid job waiting in the queue. Chapter 4 explains how to inspect a pending job.

4 Run and Monitor Slurm Jobs

4.1 Understand a Slurm Job

A Slurm job combines a request for resources with one or more commands to run. The requested resources form a job allocation. If the allocation is not immediately available, Slurm keeps the job pending in a queue. When resources become available, the job runs on one or more compute nodes.

Before submitting a job, confirm the permitted QoS and partition, estimate the required CPUs, GPUs, memory, and duration, and prepare the software environment described in Section 4.4. Request only what the program needs because larger requests can wait longer for a suitable node.

4.2 Choose a Submission Method

Use an interactive job for short experiments and troubleshooting. Use a batch job for routine analyses, training runs, or any work that should continue without an open terminal.

4.2.1 Start an Interactive Job with `srun`

Run `srun` from a remote terminal to request resources and open an interactive shell. The command may wait until the scheduler can satisfy the request. Once it starts, the terminal is attached to the allocated compute node. If `srun` is called within an existing allocation, it starts a job step inside that allocation instead of creating another allocation.

```
srun -p gpu -c2 --mem=8G --qos=gpu40g --gres=gpu:7g.40gb:1 --pty bash -l
```

Option	Description
<code>-qos=gpu40g24m</code>	Selects the QoS for the request. Use only a QoS available to your account.
<code>-c2</code>	Requests two CPU cores for the task.
<code>-mem=8G</code>	Requests 8 GB of memory for the job.
<code>-p gpu</code>	Selects the GPU partition. The partition and QoS must be compatible.
<code>-gres=gpu:7g.40gb:1</code>	Requests one GPU resource with the specified profile. Use a profile listed by CU-AIM.
<code>-pty bash -l</code>	Starts an interactive login shell after the resources are allocated.

Table 4.1: Options used in the interactive job example

To finish, run `exit` or press **Ctrl+D**. Confirm with `squeue -u your_username` that the job has ended and its resources have been released.

4.2.2 Submit a Batch Job with `sbatch`

The `sbatch` command submits a shell script, commonly saved with a `.sh` extension. Lines beginning with `#SBATCH` describe the requested resources and log files. Other lines prepare the environment and run the program. After submission, Slurm returns a job ID. Record this ID because it identifies the job in queue, accounting, log, and cancellation commands. Immediately afterward, the terminal returns, and the script starts automatically when resources become available.

The following template shows the structure of a GPU batch job. It contains both a Python-script command and a notebook command for illustration. Retain only the command required by your own job when you create a working script.

```
#!/bin/bash -l

#SBATCH --job-name=mtj
#SBATCH --partition=gpu
#SBATCH --qos=gpu40g
#SBATCH --gres=gpu:7g.40gb:1
#SBATCH --mem=20G
#SBATCH --cpus-per-task=2
#SBATCH --output=mtj-%j.out
#SBATCH --error=mtj-%j.err
#SBATCH --time=24:00:00

mkdir -p logs
module load Anaconda3
module load CUDA/11.8.0
eval "$(conda_shell.bash_hook)"
conda activate my_project

srun --output=logs/script1-%j.out \
    --error=logs/script1-%j.err \
    python -u script1.py

# if you want to execute .ipynb makesure you install papermill library
# first
srun --output=logs/script1-%j.out \
    --error=logs/script1-%j.err \
    papermill input.ipynb outputs/executed_output.ipynb

# srun allows Slurm to manage the job step and resource binding.
# This makes it easier to run jobs with multiple tasks, MPI, or multiple
# nodes.

echo "Job_completed_at_$(date)"
exit $?
```

Submit the script from a login-node terminal. Do not have to start an interactive allocation first.

```
sbatch <file_name>.sh
```

Before submission, check the following items:

- The partition, QoS, GPU profile, memory, CPU count, and time comply with current limits.
- Each `#SBATCH` directive uses a valid Slurm option for CU-AIM.
- The script loads only the required modules and activates the intended Python environment.
- Input paths exist, and the output and error paths point to writable project storage.
- The requested working directory is correct. By default, a batch job begins in the directory from which `sbatch` was called.

```
(code-server-env) [naruemon@cuaaim-frontend1 naruemon]$ sacct -u naruemonp --start=$(date -d '7 days ago' +%F) -o JobID,JobName,State,Elapsed,NodeList
JobID      JobName      State      Elapsed      NodeList
-----
44806      NAME         COMPLETED 00:00:07     cuaaim-dgx-01
44806.batch batch         COMPLETED 00:00:07     cuaaim-dgx-01
44806.extern extern        COMPLETED 00:00:07     cuaaim-dgx-01
44806.0     python       COMPLETED 00:00:01     cuaaim-dgx-01
44808      NAME         COMPLETED 00:00:07     cuaaim-dgx-01
44808.batch batch         COMPLETED 00:00:07     cuaaim-dgx-01
44808.extern extern        COMPLETED 00:00:07     cuaaim-dgx-01
44808.0     python       COMPLETED 00:00:01     cuaaim-dgx-01
```

Figure 4.1: Example batch-job monitoring information.

Section 4.5 explains how to monitor the submitted job. For multiple workloads, request the resources required by each job and respect any aggregate limits on the user, group, account, partition, or QoS. Section 5.1.2 discusses this case.

4.3 Cancel a Job

Standard Slurm cancels one or more jobs with `scancel` followed by their job IDs. The `-f` ensures the entire job is stopped, including the batch script/shell and all running job steps or subprocesses launched by it.

```
scancel -f <JOB_ID01> <JOB_ID02> <JOB_ID03>
```

4.4 Prepare the Software Environment

Each job needs a defined set of software and libraries. CU-AIM provides shared software through environment modules. Loading a module makes a particular version of Python, CUDA, or another program available in the current shell without changing the system installation.

4.4.1 Inspect and Load Environment Modules

Use these commands to inspect and manage modules:

```
module avail          # List available modules
module spider <module-name> # Search for a specific module
module load <module-name>  # Load a module
module list           # Show currently loaded modules
module unload <module-name> # Unload a module
```

```
module purge # Unload all modules
```

For example, the following commands prepare Anaconda and a CUDA toolkit in the current shell:

```
module purge
module load Anaconda3
module load CUDA/11.8.0
module list
```

Please record the module names and versions used by the project and load the same compatible set when installing packages and running jobs.

4.4.2 Check GPU and CUDA Compatibility

GPU software depends on the GPU model, NVIDIA driver, CUDA toolkit or runtime, and the machine-learning framework. After obtaining a GPU allocation, inspect the GPU and driver:

```
nvidia-smi
```

```
(base) naruemonp@cuaim-dgx-01:/data/project/naruemon/peeradon_drought$ nvidia-smi
Thu Sep 10 17:30:02 2026
```

NVIDIA-SMI 535.54.03			Driver Version: 535.54.03			CUDA Version: 12.2		
GPU	Name	Perf	Persistence-M	Bus-Id	Disp.A	Volatile	Uncorr.	ECC
Fan	Temp		Pwr:Usage/Cap		Memory-Usage	GPU-Util	Compute	M. MIG M.
0	NVIDIA A100-SXM4-40GB		On	00000000:07:00.0	Off			On

Figure 4.2: Example GPU and NVIDIA driver information reported by `nvidia-smi`.

The CUDA version shown by `nvidia-smi` indicates the newest CUDA version supported by the installed driver.

4.4.3 Create a Conda Environment

Install project-specific Python libraries in a separate Conda environment instead of the system Python installation. After loading Anaconda and any required CUDA module, create and activate an environment:

```
module load Anaconda3
module load CUDA/11.8.0
eval "$(conda shell.bash hook)"
conda create -n my_project python=3.11
conda activate my_project
```

If `conda activate` is unavailable in the current shell, initialize the Conda shell function and try again:

```
eval "$(conda shell.bash hook)"
conda activate my_project
```

4.4.4 Use a Python Virtual Environment

A Python virtual environment, or `venv`, is a lighter alternative when the required Python module is available:

```
python -m venv ~/venvs/my_project
source ~/venvs/my_project/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy pandas scikit-learn
```

Conda and `venv` are alternatives. Choose one for each project and document the environment so that another student can reproduce it.

4.4.5 Activate the Environment in a Batch Job

A batch job starts in a new, non-interactive shell. It does not reliably inherit the software configuration of a previous terminal session. Therefore, a user have to load the required modules and activate the project environment in every job script:

Listing 4.1: `example_batch_job.sh`

```
#!/bin/bash -l
#SBATCH --job-name=my_project
#SBATCH --output=logs/my_project-%j.out
#SBATCH --error=logs/my_project-%j.err
#SBATCH --partition=gpu
#SBATCH --qos=<qos>
#SBATCH --gres=gpu:1
#SBATCH --cpus=4
#SBATCH --mem=16G
#SBATCH --time=04:00:00

cd "$SLURM_SUBMIT_DIR"

module purge
module load Anaconda3
module load CUDA/11.8.0

eval "$(conda_shell.bash_hook)"
conda activate my_project

which python
python --version
python my_script.py
```

4.5 Monitor Jobs and the Queue

To monitor their pending job, the user can use `squeue` while a job is pending, running, or finished.

4.5.1 Inspect Pending and Running Jobs

The following format displays the job ID, partition, name, user, state, elapsed time, time limit, node count, pending reason or node list, and GPU information:

```
squeue -o "%.18i %.9P %.8j %.8u %.8T %.10M %.9l %.6D %R %b"
```

The full display may include other users' jobs that your account is permitted to view.

```
(code-server-env) [naruemonp@cuaaim-frontend1 naruemon]$ squeue -o "%.18i %.9P %.8j %.8u %.8T %.10M %.9l %.6D %R %b"
JOBID PARTITION NAME USER STATE TIME TIME_LIMI NODES NODELIST(REASON) TRES_PER_NODE
44977 cpu torunSTA chureera PENDING 0:00 8-00:00:00 1 (AssocMaxJobsLimit) N/A
44978 cpu torunSTA chureera PENDING 0:00 8-00:00:00 1 (AssocMaxJobsLimit) N/A
44981 cpu torunSTA chureera PENDING 0:00 8-00:00:00 1 (AssocMaxJobsLimit) N/A
45022 cpu bash hathaich PENDING 0:00 4-00:00:00 1 (AssocGrpCpuLimit) N/A
42496 cpu cgen cgen RUNNING 47-19:54:26 UNLIMITED 1 cuaaim-fat-01 N/A
44116 cpu embryo roy RUNNING 18-08:47:46 UNLIMITED 1 cuaaim-fat-01 N/A
44889 cpu metawrap pavitk RUNNING 2-02:03:52 4-00:00:00 1 cuaaim-fat-01 N/A
44933 cpu BnB anantapo RUNNING 1-02:19:52 4-00:00:00 1 cuaaim-fat-01 N/A
44957 cpu Runner pavarets RUNNING 22:12:34 4-00:00:00 1 cuaaim-fat-01 N/A
44969 cpu BnB anantapo RUNNING 56:54 4-00:00:00 1 cuaaim-fat-02 N/A
44974 cpu torunSTA chureera RUNNING 7:37:03 8-00:00:00 1 cuaaim-fat-01 N/A
44975 cpu torunSTA chureera RUNNING 5:32:03 8-00:00:00 1 cuaaim-fat-01 N/A
45018 cpu bash hathaich RUNNING 28:30 4-00:00:00 1 cuaaim-fat-01 N/A
45020 cpu bash hathaich RUNNING 25:18 4-00:00:00 1 cuaaim-fat-02 N/A
44760 gpu bash chawanp RUNNING 6-03:14:19 8-00:00:00 1 cuaaim-dgx-01 N/A
44831 gpu bash chawanp RUNNING 4-05:33:45 8-00:00:00 1 cuaaim-dgx-01 N/A
44832 gpu bash chawanp RUNNING 4-05:31:20 8-00:00:00 1 cuaaim-dgx-01 N/A
44938 gpu bash nattagit RUNNING 1-02:04:19 8-00:00:00 1 cuaaim-dgx-01 N/A
44942 gpu bash nattagit RUNNING 1-01:02:33 8-00:00:00 1 cuaaim-dgx-01 N/A
44950 gpu bash nattagit RUNNING 1-00:07:24 8-00:00:00 1 cuaaim-dgx-01 N/A
44980 gpu bash phollawa RUNNING 5:38:53 8-00:00:00 1 cuaaim-dgx-01 gres:gpu:3g.20gb:1
```

Figure 4.3: Example Slurm queue information.

```
squeue -u naruemonp
```

Replace the placeholder to show only your jobs. For a pending job, read the reason in the final output column before changing the request. Common reasons include unavailable resources, priority, account limits, or a dependency on another job. Do not attempt to cancel another user's job; contact a cluster administrator if intervention is required.

4.5.2 Inspect Job Results

Use the job ID returned by `sbatch` to inspect the final state and exit code:

```
sacct -j <job_id> --format=JobID,JobName,State,ExitCode,Elapsed
```

An exit code of 0:0 usually indicates that the batch script completed successfully. A nonzero exit code means that the script or one of its commands failed. Read the job's output and error files to identify the cause.

```
(code-server-env) [naruemonp@cuaaim-frontend1 naruemon]$ sacct -j 22449 --format=JobID,JobName,State,ExitCode,Elapsed
JobID JobName State ExitCode Elapsed
-----
22449 lstm_drou+ TIMEOUT 0:0 1-00:00:15
22449.batch batch CANCELLED 0:15 1-00:00:16
22449.extern extern COMPLETED 0:0 1-00:00:15
```

Figure 4.4: Example Slurm accounting information for a job.

Queue and accounting output commonly includes these fields:

JOBID The unique job identifier.

PARTITION The requested compute partition.

NAME The name assigned in the submission command or script.

USER The user running the job.

ST The current state code, summarized in Table 4.2.

TIME The elapsed running time. A pending job normally displays zero.

NODES The number of allocated or requested compute nodes.

NODELIST The allocated node names for a running job, or the waiting reason for a pending job.

Table 4.2: Slurm job status codes

Status	Code	Explanation
COMPLETED	CD	The job finished successfully.
COMPLETING	CG	The job has stopped running and Slurm is releasing its resources.
FAILED	F	The job ended with an error or nonzero exit code.
PENDING	PD	The job is waiting for scheduling conditions to be satisfied.
RUNNING	R	The job has resources and is executing.

```
sacct -u naruemonp --start=$(date -d '7 days ago' +%F) -o JobID,JobName,
      State,Elapsed,NodeList
```

This command lists jobs recorded for the specified user during the previous seven days. Use it to find completed, failed, canceled, or time-limited jobs when the job ID is not known.

```
(code-server-env) [naruemonp@cua1m-frontend1 naruemon]$ sacct -u naruemonp --start=$(date -d '7 days ago' +%F) -o JobID,JobName,State,Elapsed,NodeList
JobID      JobName    State    Elapsed    NodeList
-----
44806      NAME      COMPLETED 00:00:07    cua1m-dgx-01
44806.batch batch      COMPLETED 00:00:07    cua1m-dgx-01
44806.extern extern     COMPLETED 00:00:07    cua1m-dgx-01
44806.0     python    COMPLETED 00:00:01    cua1m-dgx-01
44808      NAME      COMPLETED 00:00:07    cua1m-dgx-01
44808.batch batch      COMPLETED 00:00:07    cua1m-dgx-01
44808.extern extern     COMPLETED 00:00:07    cua1m-dgx-01
44808.0     python    COMPLETED 00:00:01    cua1m-dgx-01
```

Figure 4.5: Example accounting information for completed jobs.

If the available QoS or project storage does not meet a documented research requirement, contact your supervisor or the cluster administrator. Include the expected CPU, GPU, memory, storage, and duration requirements so that the request can be evaluated.

5 Coordinate Multiple Slurm Workloads

This chapter is optional for first-time users. Read it when a project must run several independent programs or a defined sequence of programs. Begin with separate batch jobs unless the programs must share one allocation or run in a strict order.

5.1 Distinguish Jobs, Tasks, and Job Steps

5.1.1 Core Definitions

The following terms describe how Slurm organizes work:

- **Job:** A submitted resource request and its commands. The allocation may include CPUs, GPUs, memory, and a time limit.
- **Task:** A process within a job. One job may contain one or more tasks.
- **Job step:** A unit of work launched within an existing allocation, commonly with `srun`.

5.1.2 Sharing Resources Between Workloads

Slurm may apply limits to each job and to the combined activity of a user, group, account, partition, or QoS. Request the resources needed by each workload and check the current aggregate limits. If a program exceeds its assigned memory, Slurm or the operating system may terminate it. Inspect the accounting record and error log after such a failure.

5.2 Choose a Multi-Program Structure

Multiple programs can use separate allocations or share one allocation. The correct choice depends on whether they must be scheduled, monitored, and retried independently.

5.2.1 Submit Independent Jobs

This approach gives each program its own job allocation. It is appropriate when the programs do not need to start together. Each job can be monitored, retried, or canceled independently.

Listing 5.1: Sample for `job1.sh`

```
#!/bin/bash -l
#SBATCH --job-name=mtj01
```

```
#SBATCH --partition=gpu
#SBATCH --qos=gpu40g24m
#SBATCH --gres=gpu:7g.40gb:1
#SBATCH --mem=20G
#SBATCH --cpus-per-task=2
#SBATCH --output=mtj-%j.out
#SBATCH --error=mtj-%j.err
#SBATCH --time=24:00:00

python script1.py
```

Listing 5.2: Sample for job2.sh

```
#!/bin/bash -l
#SBATCH --job-name=mtj02
#SBATCH --partition=gpu
#SBATCH --qos=gpu40g24m
#SBATCH --gres=gpu:7g.40gb:1
#SBATCH --mem=20G
#SBATCH --cpus-per-task=2
#SBATCH --output=mtj-%j.out
#SBATCH --error=mtj-%j.err
#SBATCH --time=24:00:00

python script2.py
```

Submit the scripts separately with `sbatch job1.sh` and `sbatch job2.sh`. Slurm schedules each job according to its request and the applicable limits.

5.2.2 Run Concurrent Steps Within One Job

This approach requests one allocation large enough for all programs. Each `srun` command starts a job step within that allocation. Use it only when the parent allocation provides the combined CPUs, memory, and GPUs required by all steps.

Listing 5.3: combined_job.sh

```
#!/bin/bash -l
#SBATCH --job-name=mtj
#SBATCH --partition=gpu
#SBATCH --qos=gpu40g24m
#SBATCH --gres=gpu:7g.40gb:2
#SBATCH --mem=20G
#SBATCH --cpus-per-task=2
#SBATCH --output=mtj-%j.out
#SBATCH --error=mtj-%j.err
mkdir -p logs

# Launch task 1 in the background
srun --exclusive \
    --ntasks=1 \
    --cpus-per-task=1 \
    --gres=gpu:7g.40gb:1 \
    --mem=10G \
```

```

--output=logs/script1-%j.out \
--error=logs/script1-%j.err \
python -u script1.py &

pid1=$!

# Launch task 2 in the background
srun --exclusive \
    --ntasks=1 \
    --cpus-per-task=1 \
    --gres=gpu:7g.40gb:1 \
    --mem=10G \
    --output=logs/script2-%j.out \
    --error=logs/script2-%j.err \
    python -u script2.py &

pid2=$!

# Wait for both tasks and preserve their exit statuses
wait "$pid1"
status1=$?

wait "$pid2"
status2=$?

if [[ $status1 -ne 0 || $status2 -ne 0 ]]; then
    echo "At least one task failed."
    exit 1
fi

echo "Both tasks completed successfully."

```

The ampersand (&) allows the two shell commands to overlap. Actual concurrent execution still depends on sufficient resources and valid Slurm step requests. The final `wait` commands keep the batch job active until both programs finish.

5.3 Run a Numbered Series of Programs

Use a job array when numbered programs are independent. Use a shell loop within one batch job when they must run in a strict order. A concurrency limit on a job array controls how many elements run at once, but it does not guarantee numerical execution order.

5.3.1 Limit a Job Array to One Running Element

The following excerpt illustrates a ten-element array with a concurrency limit of one:

Listing 5.4: The array and task-selection lines in `test_sequential_job.sh`

```

# Create ten array elements; at most one may run at a time.
#SBATCH --array=1-10%1

i="$SLURM_ARRAY_TASK_ID"
python -u "script${i}.py"

```

The range 1-10 creates ten array elements. The suffix %1 allows no more than one element to run at a time. Other elements remain pending until the running element finishes and all scheduling conditions are met. Slurm chooses which eligible element starts next, so this pattern must not be used when one program depends on the output of the previous index.

	JOBID	PARTITION	NAME	USER	ST	TIME	NODES
	398545	cpu	dgc_n_mtl	naruemon	PD	0:00	1
	398543	cpu	dgc_n_mtl	naruemon	PD	0:00	1
	398542	cpu	dgc_n_mtl	naruemon	PD	0:00	1
	398541_[2-39%1]	gpu	dgc_n_mtl	naruemon	PD	0:00	1
	398544_[0-4%1]	gpu	dgc_n_mtl	naruemon	PD	0:00	1
	398541_1	gpu	dgc_n_mtl	naruemon	R	0:17	1

Figure 5.1: A Slurm queue showing a running array element and pending elements constrained by an array concurrency limit.

The complete included example currently defines three array elements. Adjust its array range in a working copy to match the number of programs in your project:

Listing 5.5: A Slurm job array with limited concurrency

```
#!/bin/bash -l
#The array creates jobs the %1 means that only one array task runs at a
time
#SBATCH --job-name=test-array
#SBATCH --partition=gpu
#SBATCH --qos=gpu40g
#SBATCH --gres=gpu:7g.40gb:1
#SBATCH --mem=20G
#SBATCH --cpus-per-task=2
#SBATCH --time=24:00:00
#SBATCH --array=1-3%1
#SBATCH --output=mtj-%A_%a.out
#SBATCH --error=mtj-%A_%a.err

set -e

mkdir -p logs

module load Anaconda3
module load CUDA/11.8.0

eval "$(conda_ shell.bash_ hook)"
conda activate my_project

i="$SLURM_ARRAY_TASK_ID"

echo "Running_ script${i}.py"

srun --exclusive \
    --ntasks=1 \
    --cpus-per-task=1 \
```

```

--gres=gpu:7g.40gb:1 \
--mem=10G \
--output="logs/script${i}-${SLURM_ARRAY_JOB_ID}_${i}.out" \
--error="logs/script${i}-${SLURM_ARRAY_JOB_ID}_${i}.err" \
python -u "script${i}.py"

echo "Finished script${i}.py"

```

Each element receives a `SLURM_ARRAY_TASK_ID`, which selects the corresponding program. The array job ID and element index in each log filename make the runs individually traceable.

5.3.2 Run a Strict Sequence Within One Batch Job

The next example submits one batch job and loops through ten programs within the same allocation:

Listing 5.6: A strict sequence within one batch job

```

#!/bin/bash -l
#This will create only one job but multiple tasks inside
#SBATCH --job-name=loop_task
#SBATCH --partition=gpu
#SBATCH --qos=gpu40g
#SBATCH --gres=gpu:7g.40gb:1
#SBATCH --mem=20G
#SBATCH --cpus-per-task=2
#SBATCH --output=mtj-%j.out
#SBATCH --error=mtj-%j.err
#SBATCH --time=24:00:00

set -e

mkdir -p logs

module load Anaconda3
module load CUDA/11.8.0

eval "$(conda shell.bash_hook)"
conda activate my_project

for i in {1..10}; do
    echo "Running script${i}.py"

    srun --exclusive \
        --ntasks=1 \
        --cpus-per-task=1 \
        --gres=gpu:7g.40gb:1 \
        --mem=10G \
        --output="logs/script${i}-%j.out" \
        --error="logs/script${i}-%j.err" \
        python -u "script${i}.py"

    echo "Finished script${i}.py"
done

```

Each `srun` command finishes before the loop starts the next iteration, so the numerical order is preserved. The script uses shell error handling to stop after a failed step. All steps share the time limit of the parent allocation, so estimate the total duration of the complete sequence.

The two approaches can be summarized as follows:

Table 5.1: Comparison of numbered Slurm execution patterns

Property	Job array	Single job loop
Slurm records	One array with individually traceable elements	One batch job with several job steps
Execution order	Not guaranteed by the %1 concurrency limit	Preserved by the blocking <code>for</code> loop
Failure handling	Each element has its own state	The script can stop the loop after a failed step
Use when	Programs are independent	Programs must run in order within one allocation

Submit and monitor the array version with:

```

sbatch ssh_example/test_sequential_job.sh
squeue -u "$USER"

```

Use the array version for independent programs that benefit from separate states and logs. Use the single-job loop when order matters or all programs must share one allocation.

6 Use Singularity Containers

Use a container only when the module and Python-environment workflow in Section 4.4 cannot provide compatible dependencies. A Singularity image is a file containing an isolated software environment. It can provide newer application libraries, but it still uses the host operating-system kernel and, for GPU work, the host NVIDIA driver.

6.1 Obtain a Container Image

First confirm that Singularity is available on CU-AIM and that the selected image is permitted by cluster policy. The following commands download a Docker or Open Container Initiative (OCI) image and save it as a Singularity Image Format (SIF) file:

```
PROJECT=/data/project/narueemon/<project_name>

export SINGULARITY_CACHEDIR="$PROJECT/.singularity-cache-$USER"
export SINGULARITY_TMPDIR="$PROJECT/.singularity-tmp-$USER"
export TMPDIR="$PROJECT/tmp-$USER"

mkdir -p "$SINGULARITY_CACHEDIR" "$SINGULARITY_TMPDIR" "$TMPDIR"

singularity pull \
  "$PROJECT/<singularity_name>.sif" \
  docker://<pull_url>

singularity cache clean --force
```

The environment variables redirect download and temporary files from the limited home directory to project storage. The output path in `singularity pull` determines where the final SIF file is stored.

Use an image from a trusted registry. Record the registry address, image name, and immutable version or digest in the project documentation so that the environment can be reproduced.

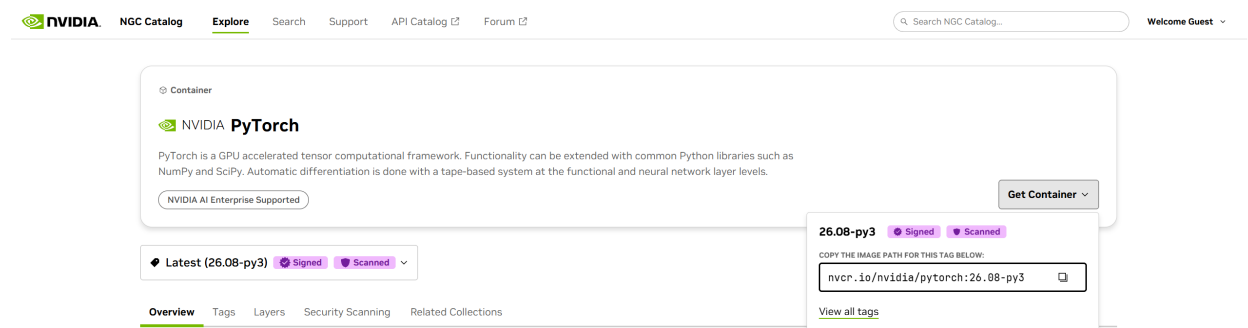


Figure 6.1: Example container image source.

6.2 Create a Python Environment in the Container

The container image is normally read-only. The following workflow makes the project directory visible inside the container at `/workspace` and stores a writable Python virtual environment there. The environment remains available after the container command ends.

6.2.1 Define Paths and Install Dependencies

Replace the project, image, and environment placeholders before running any command. Keep the same definitions throughout one terminal session.

The `-nv` option exposes the host NVIDIA driver and allocated GPU to the container. Use it only within a GPU allocation when the program requires a GPU.

```
# Define paths and retain these variables for the session.
PROJECT=/data/project/naruemon/your_project
IMAGE=$PROJECT/containers/your_container.sif
ENV_TAG=your_environment
VENV=/workspace/.venv-$ENV_TAG

# Verify GPU access inside the container.
singularity exec --nv \
  --bind "$PROJECT:/workspace:rw" \
  "$IMAGE" \
  nvidia-smi

# Create the virtual environment.
singularity exec --nv \
  --bind "$PROJECT:/workspace:rw" \
  "$IMAGE" \
  python -m venv "$VENV"

# Install PyTorch from the selected CUDA wheel index.
singularity exec --nv \
  --bind "$PROJECT:/workspace:rw" \
  "$IMAGE" \
  "$VENV/bin/python" -m pip install --no-cache-dir \
  torch torchvision \
  --index-url https://download.pytorch.org/whl/cu126
```

```
# Install the remaining project dependencies.
singularity exec --nv \
  --bind "$PROJECT:/workspace:rw" \
  "$IMAGE" \
  "$VENV/bin/python" -m pip install --no-cache-dir \
  -r /workspace/requirements.txt

# Uninstall packages listed in the same requirements file.
singularity exec --nv \
  --bind "$PROJECT:/workspace:rw" \
  "$IMAGE" \
  "$VENV/bin/python" -m pip uninstall -y \
  -r /workspace/requirements.txt

# Completely reset the project-specific virtual environment.
rm -rf -- "$PROJECT/.venv-$ENV_TAG"
```

The `-bind` option maps the project directory to `/workspace` inside the container. The selected PyTorch package must support the allocated GPU and the host NVIDIA driver.

The final two groups of commands in the example are maintenance operations, not normal installation steps. The `uninstall` command removes packages, and the `reset` command permanently deletes the project-specific virtual environment. Run either operation only when you intentionally need to repair or recreate that environment, and verify the resolved project path first.

6.3 Run a Container in a Batch Job

A containerized program uses the same Slurm resource directives as any other batch job. The job must request a GPU when `singularity exec --nv` is used. Replace the project-specific paths in the example before submission.

```
#!/bin/bash -l
#SBATCH --job-name=mtj
#SBATCH --partition=gpu
#SBATCH --qos=gpu40g24m
#SBATCH --gres=gpu:7g.40gb:2
#SBATCH --mem=20G
#SBATCH --cpus-per-task=2
#SBATCH --output=mtj-%j.out
#SBATCH --error=mtj-%j.err
mkdir -p logs

set -euo pipefail

PROJECT=/data/project/narueemon/your_project
IMAGE=$PROJECT/containers/your_container.sif
ENV_TAG=your_environment
VENV=/workspace/.venv-$ENV_TAG

singularity exec --nv \
  --pwd /workspace \
```

```
--bind "$PROJECT:/workspace:rw" \  
"$IMAGE" \  
"$VENV/bin/python" \  
/workspace/script1.py
```

6.4 Run a Container Interactively

First request an interactive allocation with a permitted partition, QoS, and GPU profile:

```
srun -p gpu -c2 --mem=8G --qos=gpu40g24m --gres=gpu:7g.40gb:1 --pty bash  
-l
```

After the allocation starts, define the project, image, and environment paths:

```
PROJECT=/data/project/naruemon/your_project  
IMAGE=$PROJECT/containers/your_container.sif  
ENV_TAG=your_environment  
VENV=/workspace/.venv-$ENV_TAG
```

Run the program inside the container. Retain `-nv` for GPU work; omit it for CPU-only work:

```
singularity exec --nv \  
--pwd /workspace \  
--bind "$PROJECT:/workspace:rw" \  
"$IMAGE" \  
"$VENV/bin/python" \  
/workspace/script1.py
```

“When life leaves us blind, love keeps us kind.”

— Chester Bennington